

Car QR Showcase - Technical Documentation

Project Overview

A web-based car dealership management system with QR code generation for vehicle identification and customer engagement tracking.

****Live Application**:** <https://m047t8wjv46u.space.minimax.io>

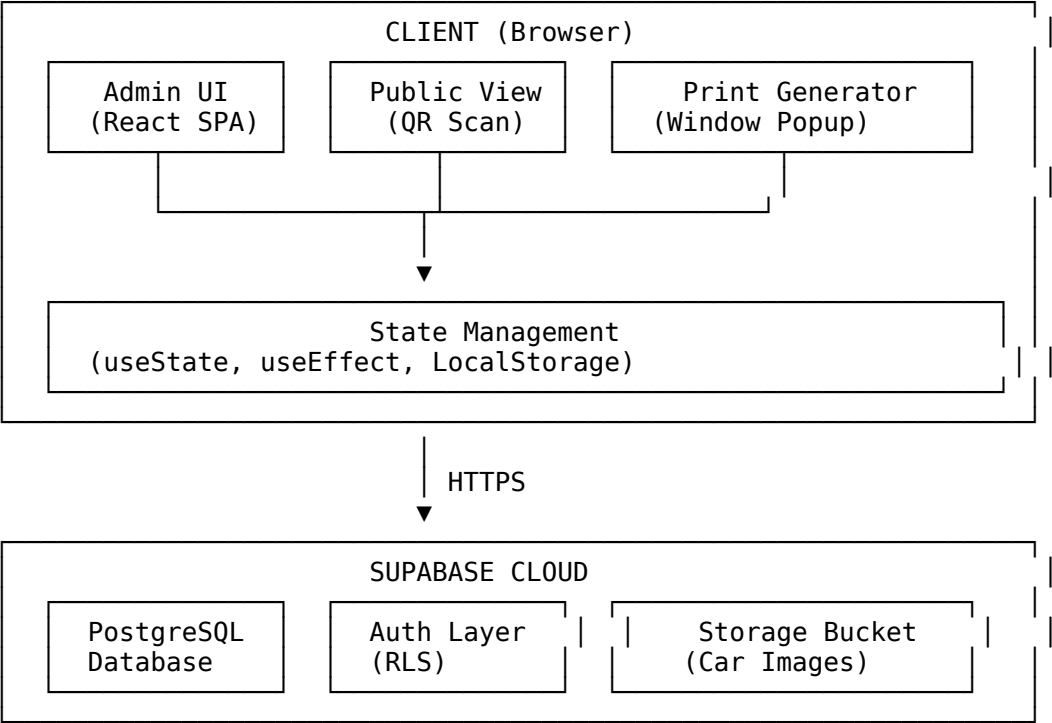
Table of Contents

- 1. [System Architecture](#system-architecture)
- 2. [Technology Stack](#technology-stack)
- 3. [Database Design](#database-design)
- 4. [Core Features Implementation](#core-features-implementation)
- 5. [API Design](#api-design)
- 6. [Component Architecture](#component-architecture)
- 7. [Security Considerations](#security-considerations)
- 8. [Print System Design](#print-system-design)
- 9. [Development Workflow](#development-workflow)
- 10. [Deployment Strategy](#deployment-strategy)

System Architecture

High-Level Architecture

...



...

Data Flow

...

User Action → React Event Handler → State Update → API Call → Supabase → Response → UI Update

...

Technology Stack

Frontend

Technology	Version	Purpose
React	18.3.x	UI framework
TypeScript	5.6.x	Type safety
Vite	6.2.x	Build tool
Tailwind CSS	3.4.x	Styling
Lucide React	0.364.x	Icons
QRCode.react	4.2.x	QR generation

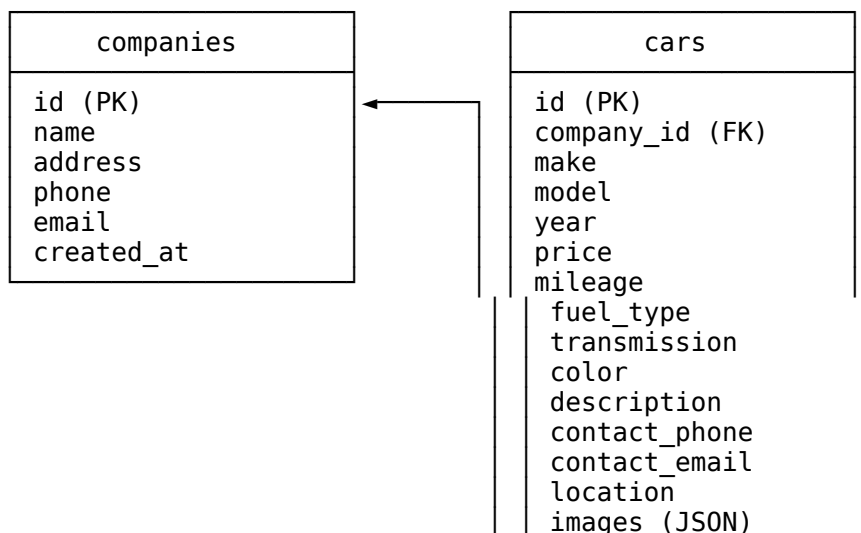
Backend

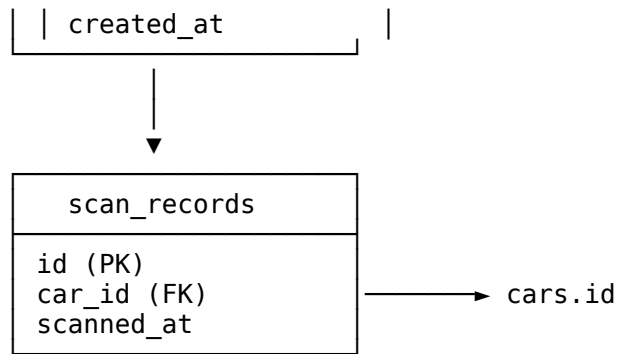
Technology	Purpose
Supabase	Database + Auth + Storage
PostgreSQL	Relational database
Row Level Security	Data access control

Database Design

Entity Relationship Diagram

...





```

### ### Table Definitions

#### #### companies

```

```sql
CREATE TABLE companies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  address TEXT,
  phone TEXT,
  email TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

```

cars

```

```sql
CREATE TABLE cars (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 company_id UUID REFERENCES companies(id),
 make TEXT NOT NULL,
 model TEXT NOT NULL,
 year INTEGER,
 price TEXT,
 mileage TEXT,
 fuelType TEXT,
 transmission TEXT,
 color TEXT,
 description TEXT,
 contactPhone TEXT,
 contactEmail TEXT,
 location TEXT,
 images JSONB DEFAULT '[]',
 created_at TIMESTAMPTZ DEFAULT NOW()
);

```

#### #### scan\_records

```

```sql
CREATE TABLE scan_records (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```
    car_id UUID REFERENCES cars(id),
    scanned_at TIMESTAMPTZ DEFAULT NOW()
);
```

Indexes

```
```sql
CREATE INDEX idx_cars_company ON cars(company_id);
CREATE INDEX idx_scans_car ON scan_records(car_id);
CREATE INDEX idx_scans_date ON scan_records(scanned_at);
```
```

Core Features Implementation

1. Company Management

```
```typescript
interface CompanyData {
 id: string
 name: string
 address: string
 phone: string
 email: string
 created_at: string
}

// CRUD Operations
- Create: POST /companies
- Read: GET /companies
- Update: PATCH /companies/:id
- Delete: DELETE /companies/:id (cascade)
```
```

2. Car Management

```
```typescript
interface CarData {
 id: string
 company_id: string
 make: string
 model: string
 year: number
 price: string
 mileage: string
 fuelType: string
 transmission: string
 color: string
 description: string
 contactPhone: string
 contactEmail: string
 location: string
 images: string[] // Base64 or URL
 created_at: string
}
```

...

### ### 3. QR Code Generation

```
``typescript
// URL Structure
const getCarUrl = (carId: string) => {
 const baseUrl = window.location.origin
 return `${baseUrl}?car=${carId}`
}

// QR Display Component
<QRCodeSVG value={getCarUrl(car.id)} size={60} level="H" />

// Print QR API
https://api.qrserver.com/v1/create-qr-code/?
size=120x120&data=${encodeURIComponent(url)}
``
```

### ### 4. Scan Tracking

```
``typescript
// Track when QR is scanned
async function recordScan(carId: string) {
 await supabase.from('scan_records').insert({
 car_id: carId,
 scanned_at: new Date().toISOString()
 })
}

// Fetch on page load
async function fetchCarById(carId: string) {
 const [carRes, companiesRes] = await Promise.all([
 supabase.from('cars').select('*').eq('id', carId).single(),
 supabase.from('companies').select('*')
])
 // Set state for public view
}
``
```

### ### 5. Image Management

```
``typescript
// Two image sources supported:
// 1. Base64 (uploaded directly)
// 2. URL (external link)

// Display in grid
{images.map((img, idx) => (

))}

// Lightbox view
<div onClick={() => setLightboxImage(img)}>

</div>
``
```

---

## ## API Design

### ### Supabase REST API

```

```
GET    /companies          - List all companies
POST   /companies          - Create company
PATCH /companies?id=eq.X  - Update company
DELETE /companies?id=eq.X  - Delete company

GET    /cars               - List all cars
GET    /cars?company_id=eq.X - List cars by company
POST   /cars               - Create car
PATCH /cars?id=eq.X       - Update car
DELETE /cars?id=eq.X       - Delete car

GET    /scan_records       - List all scans
POST   /scan_records       - Record new scan
```
```

### ### Query Parameters

```
```typescript
// Filter by company
supabase.from('cars').select('*').eq('company_id', companyId)

// Order by date
supabase.from('cars').select('*').order('created_at', { ascending: false })

// Single record
supabase.from('cars').select('*').eq('id', carId).single()
```
```

---

## ## Component Architecture

### ### Main Application Flow

```

```
App Component
├── Auth State Check
│   ├── Not Logged In → Login Screen
│   └── Logged In → Admin Dashboard
├── Public View (URL param: ?car=X)
│   └── Car Details + Scan Recording
└── Admin Dashboard
    ├── Header (Stats + Actions)
    ├── Company List
    │   └── Expanded Company
    │       ├── Add Car Button
    │       └── Print Barcodes Button
```

```

    └── View Report Button
        └── Car Cards Grid
            └── Car Actions (View, Edit, Delete)

└── Modals
    ├── Company Form
    ├── Car Form
    └── Report Modal

└── Utilities
    ├── Image Lightbox
    └── Print Window
...

```

State Management

```

```typescript
// Core State
const [isLoggedIn, setIsLoggedIn] = useState(false)
const [companies, setCompanies] = useState<CompanyData[]>([])
const [cars, setCars] = useState<CarData[]>([])
const [scanRecords, setScanRecords] = useState<ScanRecord[]>([])

// UI State
const [editingCompany, setEditingCompany] = useState<CompanyData | null>(null)
const [editingCar, setEditingCar] = useState<CarData | null>(null)
const [viewingCarId, setViewingCarId] = useState<string | null>(null)
const [expandedCompany, setExpandedCompany] = useState<string | null>(null)

// Print State
const [qrStyle, setQrStyle] = useState<'standard' | 'luxury'>('standard')
```

```

Security Considerations

Row Level Security (RLS)

```

```sql
-- Enable RLS
ALTER TABLE companies ENABLE ROW LEVEL SECURITY;
ALTER TABLE cars ENABLE ROW LEVEL SECURITY;
ALTER TABLE scan_records ENABLE ROW LEVEL SECURITY;

-- Public read access (for QR scanning)
CREATE POLICY "Public can read all" ON companies FOR SELECT USING (true);
CREATE POLICY "Public can read cars" ON cars FOR SELECT USING (true);
CREATE POLICY "Public can insert scans" ON scan_records FOR INSERT WITH CHECK
(true);

-- Admin write access (via authenticated sessions - future enhancement)
CREATE POLICY "Authenticated can manage" ON companies
 FOR ALL USING (auth.role() = 'authenticated');
```

```

Client-Side Security

```

```typescript
// Password stored in localStorage (for demo purposes)
const STORAGE_KEYS = {
 ADMIN_PASSWORD: 'carqr_admin_password',
 IS_LOGGED_IN: 'carqr_logged_in'
}

// Note: In production, use Supabase Auth for proper security
```

```

Input Validation

```

```typescript
// Required fields check
if (!company.name.trim()) {
 alert('Company name is required')
 return
}

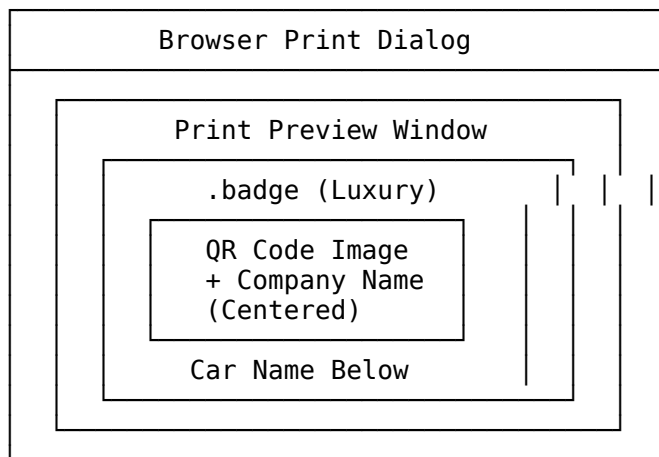
// XSS prevention (React handles escaping automatically)
// Never use dangerouslySetInnerHTML with user input
```

```

Print System Design

Print Window Architecture

...



...

Print Styles

Luxury Style

```

```css
.badge {
 background: linear-gradient(135deg, #1a1a2e 0%, #16213e 100%);
 border-radius: 20px;
}

```

```
padding: 25px;
min-width: 240px;
box-shadow: 0 10px 40px rgba(0,0,0,0.2);
}
```

```
.company-name {
 color: #c9a227; /* Gold */
 font-size: 12px;
 font-weight: bold;
 position: absolute;
 top: 50%;
 left: 50%;
 transform: translate(-50%, -50%);
 background: rgba(26,26,46,0.9);
 padding: 6px 10px;
 border: 1px solid #c9a227;
}
\`\`\`
```

#### #### Standard Style

```
\`\`\`css
.container {
 width: 260px;
 margin: 30px auto;
 padding: 20px;
 border: 3px solid #000;
}

.company-name {
 font-size: 12px;
 color: #fff;
 font-weight: bold;
 background: #000;
 position: absolute;
 top: 50%;
 left: 50%;
 transform: translate(-50%, -50%);
 padding: 4px 10px;
 border: 2px solid #000;
}
\`\`\`
```

#### ### Print Trigger Mechanism

```
\`\`\`typescript
// Auto-print on image load
const printSingleCarQR = (carId: string) => {
 const printWindow = window.open('', '_blank', 'width=450,height=500')

 printWindow.document.write(`
 <html>
 <body>

 </body>
 </html>
 `)
}
```

```

 printWindow.document.close()
 }

 // Alternative: Delayed print
 setTimeout(() => printWindow.print(), 600)
  ```

```

Print Media Queries

```

```css
@media print {
 body {
 background: #fff;
 -webkit-print-color-adjust: exact;
 print-color-adjust: exact;
 }

 .container {
 margin: 10px auto;
 padding: 15px;
 border-width: 2px;
 box-shadow: none;
 }
}
```

```

Development Workflow

1. Project Setup

```

```bash
Create Vite React TypeScript project
npm create vite@latest car-qr-showcase -- --template react-ts

Install dependencies
npm install
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p

Install additional packages
npm install @supabase/supabase-js qrcode.react lucide-react
```

```

2. Development Cycle

```

```
Edit Code → Build → Test → Deploy → Verify
 ↑ |
 └──────┘
```

```

3. State Management Pattern

```

```typescript

```

```
// 1. Define state
const [state, setState] = useState<Type>(initialValue)
```

```
// 2. Update state
setState(newValue)
```

```
// 3. Effect on state change
useEffect(() => {
 // Side effects
}, [dependency])
``,`
```

#### ### 4. Error Handling

```
```typescript
// Try-catch for async operations
async function fetchData() {
  try {
    const { data, error } = await supabase.from('cars').select('*')
    if (error) throw error
    setCars(data || [])
  } catch (err) {
    console.error('Fetch error:', err)
    alert('Failed to load data')
  }
}
}
```

```
// Null checks
const car = cars.find(c => c.id === carId)
if (!car) return
``,`
```

Deployment Strategy

Build Process

```
```bash
Development
npm run dev

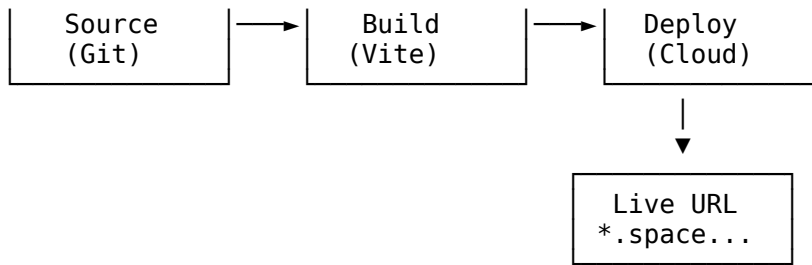
Production build
npm run build

Output in dist/ folder
dist/
├── index.html
├── assets/
│ ├── index-*.css
│ └── index-*.js
``,`
```

##### ### Deployment Flow

...

┌──────────┐ ┌──────────┐ ┌──────────┐



...

### ### Post-Deployment Verification

```

```typescript
// Verify build output
- dist/index.html exists
- dist/assets/*.css exists
- dist/assets/*.js exists

// Test key features
- Company creation
- Car management
- QR generation
- Print functionality
- Public scan view
```

```

---

## ## Performance Considerations

### ### Code Splitting

```

```typescript
// Lazy load print functionality
const printBarcodes = React.useCallback((companyId: string) => {
  // Only load when needed
}, [])
```

```

### ### Image Optimization

```

```typescript
// Limit image size
const MAX_IMAGES = 10

// Use thumbnails for list view
const thumbnail = images.slice(0, 10).map(img => (
  <img src={img} className="w-12 h-12 object-cover rounded" />
))
```

```

### ### Database Optimization

```

```typescript
// Select only needed fields
supabase.from('cars').select('id, make, model, year, price')
```

```

```

// Use indexes
CREATE INDEX idx_cars_company ON cars(company_id);

// Paginate large datasets (future)
supabase.from('cars').select('*').range(0, 50)
```

---

## Future Enhancements

### Potential Improvements

1. Authentication: Replace localStorage with Supabase Auth
2. Real-time: Add Supabase Realtime for live updates
3. Analytics: Dashboard with charts (recharts is already installed)
4. Bulk Operations: Batch import/export CSV
5. PWA Support: Add service worker for offline access
6. Custom Domains: Point to custom domain

### Scalability

```typescript
// Current: Single-tenant per Supabase project
// Future: Multi-tenant with company_id isolation

// Add pagination
const [page, setPage] = useState(1)
supabase.from('cars')
 .select('*')
 .eq('company_id', companyId)
 .range((page - 1) * 20, page * 20)
```

---

## Appendix: File Structure

```
car-qr-showcase/
├── src/
│ ├── App.tsx # Main application (960 lines)
│ ├── lib/
│ │ └── supabase.ts # Database client
│ ├── index.css # Tailwind imports
│ └── main.tsx # React entry point
├── public/
│ └── vite.svg
├── dist/ # Production build output
├── package.json
├── tsconfig.json
├── vite.config.ts
├── tailwind.config.js
├── postcss.config.js
└── PROJECT_SUMMARY.md # User-facing documentation
```

```

****Document Version**:** 1.0
****Last Updated**:** June 2026
****Author**:** MiniMax Agent